



UNIVERSIDAD
CATÓLICA
BOLIVIANA

INFORME PRIMERA EVALUACIÓN

ASIGNATURA INTERNET DE LAS COSAS

GRUPO N.º

INTEGRANTES

REPOSITORIO GITHUB

DOCENTE

CARRERA

FECHA DE ENTREGA

GESTIÓN

1. Alvaro Torrez

2. Karen Vargas

https://github.com/alvaro-tc/iot_dashboard

Ing. Pamela Valenzuela

Ingeniería de Sistemas

11/09/2026

II/2026

**LA PAZ – BOLIVIA
2026**

INFORME – PRIMERA EVALUACIÓN

Proyecto: *Primera Evaluación IoT* — plataforma cliente-servidor que recibe, almacena y visualiza en tiempo real series matemáticas de aproximación generadas por dispositivos ESP32-S3 (o por un simulador web) y enviadas por MQTT. Repositorio: https://github.com/alvaro-tc/iot_dashboard.

Contenido

1. Introducción	4
2. Objetivos	5
Objetivo general	5
Objetivos específicos	5
3. Diseño de la solución cliente-servidor	5
Arquitectura y componentes	5
Flujo de datos	6
Identificación del usuario de cada cliente	6
Número de registro (n)	6
4. Diseño de la base de datos	6
Nombre y tecnología	6
Tablas, campos y tipos	6
Restricciones y reglas aplicadas	8
Justificación de campos y tipos	8
Diagrama entidad-relación	9
5. Desarrollo de la aplicación	9
Tecnologías, librerías y herramientas	9
Implementación del cliente	9
Implementación del servidor	10
Implementación de las tres series matemáticas	11
Inserción de datos en la base de datos	11
Lectura o recuperación de datos	12
Otras funciones importantes	12
6. Desarrollo del dashboard	13
Diseño y herramientas	13
Visualización de datos por usuario y su error	14
Gráficos	14
Código relevante del dashboard	17
Actualización en tiempo real	18
Interactividad	19
7. Pruebas y resultados	20
Comunicación cliente-servidor	20
Generación y almacenamiento de las tres series	21
Registros asociados a cada usuario y al valor n	21

Capturas del dashboard con datos y error	23
Análisis de resultados	24
8. Conclusiones	24
9. Anexos y entregables	25

1. Introducción

Problema. En un entorno IoT varios dispositivos generan datos de forma continua y concurrente. Se necesita un sistema que identifique a qué usuario pertenece cada dato, lo valide, lo persista de manera consistente y permita analizarlo — en este caso, cuán rápido una serie matemática converge a su valor real — sin que un dispositivo pueda escribir en nombre de otro.

Solución propuesta. Se desarrolló un monorepo (pnpm workspaces) con cuatro piezas: el **cliente** (ESP32-S3 con firmware PubSubClient o un simulador web que usa exactamente el mismo camino), un **broker MQTT** Mosquitto con credenciales y ACL por dispositivo, un **servidor** Node.js/Express que actúa como puente MQTT → PostgreSQL → WebSocket, y un **dashboard** React que muestra los datos de cada usuario y su error en tiempo real. El cliente publica en el topic `telemetry/{userId}/{seriesKey}` un JSON con el número de iteración n y el valor aproximado; el servidor calcula el error absoluto $|x_n - x_{\text{real}}|$, agrupa las muestras en **sesiones** y notifica al dashboard.

Series seleccionadas para la evaluación. El sistema implementa siete series en una única fuente de verdad (`packages/shared/src/series.ts`); para la evaluación se eligen tres, las que envía el ESP32 de prueba:

Serie	Fórmula	Valor real	Error
Leibniz–Gregory	$\pi = \sum_{k \geq 1} \frac{4(-1)^{k+1}}{2k-1}$	3.1415926536	$\sim \frac{1}{n}$, oscilante
Producto de Wallis	$\pi = 2 \prod_{k \geq 1} \frac{2k}{2k-1} \cdot \frac{2k}{2k+1}$	3.1415926536	$\sim \frac{\pi}{4n}$, monótono
Problema de Basilea	$\frac{\pi^2}{6} = \sum_{k \geq 1} \frac{1}{k^2}$	1.6449340668	$\sim \frac{1}{n}$, monótono

- **Leibniz–Gregory (π).** Serie alternada: cada término cambia de signo, por lo que el valor oscila por encima y por debajo de π . Es el ejemplo clásico de convergencia lenta y sirve para validar que el dashboard muestra correctamente oscilaciones (el submuestreo del gráfico conserva mínimos y máximos para no aplanarlas).
- **Producto de Wallis (π).** A diferencia de las otras, es un **producto infinito**: el acumulador inicia en 1 y se multiplica en cada iteración. Permite comparar dos métodos distintos que aproximan la misma constante y ver cuál converge más rápido.
- **Problema de Basilea ($\pi^2/6$).** Suma de términos positivos: la aproximación crece monótonamente hacia el valor real. Su error, visto en escala logarítmica, muestra una curva suave y decreciente, útil para interpretar la tasa de convergencia.

Adicionalmente están implementadas la armónica alternada ($\ln 2$), Euler–Mascheroni (γ), la suma de impares al cuadrado ($\pi^2/8$) y la constante de Catalan (G , con error $1/n^2$, como contraste de convergencia rápida).

Interacción de componentes. El ESP32 (o el simulador) publica por MQTT → Mosquitto verifica usuario/contraseña y ACL → el servidor, suscrito a `telemetry/+/*`, valida el payload, calcula el error y guarda la muestra en PostgreSQL dentro de una transacción → tras el `COMMIT` emite eventos por WebSocket → el dashboard actualiza tablas y gráficos sin recargar.

2. Objetivos

Objetivo general

Desarrollar una aplicación cliente-servidor IoT que genere tres series matemáticas de aproximación desde clientes identificados, almacene cada valor con su número de registro y error en una base de datos relacional, y los visualice en un dashboard en tiempo real por usuario.

Objetivos específicos

- Implementar el cliente (firmware ESP32 y simulador web) que publique por MQTT el valor de la iteración n de la serie elegida, identificado por usuario y dispositivo.
- Implementar el servidor que reciba, valide (zod), procese e ingrese las muestras, calculando el error absoluto respecto del valor real.
- Implementar las series de Leibniz, Wallis y Basilea (y cuatro adicionales) como funciones puras compartidas entre servidor, simulador y datos de prueba.
- Diseñar una base de datos PostgreSQL normalizada que registre usuario, dispositivo, sesión, número de iteración, tipo de serie, valor y error, con restricciones de integridad.
- Desarrollar un dashboard web con gráficos de valor y error (escala logarítmica), filtros, línea de tiempo, exportación CSV y actualización en tiempo real por WebSocket.
- Asegurar la comunicación: autenticación JWT en la web, credenciales y ACL por dispositivo en el broker.

3. Diseño de la solución cliente-servidor

Arquitectura y componentes

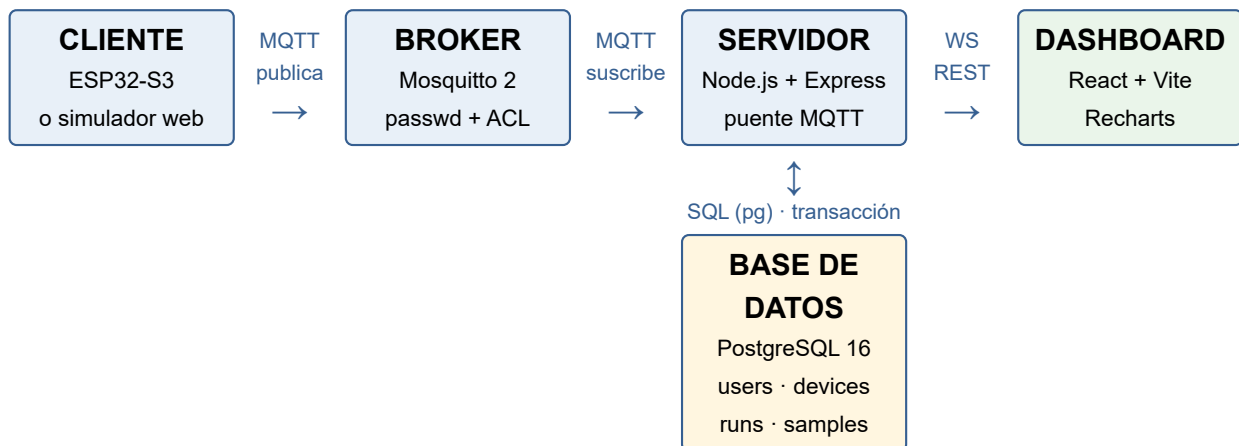


Figura 1: Diagrama de bloques de la arquitectura.

- **Cliente:** el ESP32-S3 calcula la serie con `double` y publica cada iteración. Los usuarios sin placa usan el **simulador web**, que se ejecuta en el servidor pero publica por MQTT igual que un dispositivo, recorriendo el mismo camino.
- **Broker (Mosquitto):** `allow_anonymous false`; cada dispositivo tiene usuario/contraseña propios y una ACL que solo le permite escribir en `telemetry/{su userId}/+`.
- **Servidor (apps/api):** Express 5 (REST), `mqtt` (puente), `pg` (SQL directo, sin ORM), `ws` (tiempo real), JWT + `bcrypt` (autenticación).

- **Base de datos:** PostgreSQL 16 en Docker (puerto 5433).
- **Dashboard (apps/web):** React 18 + TypeScript + Tailwind + Recharts, con vistas de administrador y de cliente.

Flujo de datos

1. **Generación:** el cliente calcula el término k con `computeTerm(serie, k, acc)` y arma el payload `{"iteration": n, "value": 3.1415926536, "deviceId": "esp32-..."}`.
2. **Envío:** publica en `telemetry/2/pi_leibniz`. Mosquitto rechaza la conexión si el token no es válido y descarta la publicación si el topic no corresponde al dueño.
3. **Recepción:** el servidor (suscrito a `telemetry/+/+`) extrae `userId` y `seriesKey` del topic y valida el JSON con zod. Encola la muestra en una cola **por usuario** para procesarlas en orden de llegada.
4. **Procesamiento:** en una transacción bloquea la fila del usuario, verifica dispositivo (existente, no revocado, del mismo usuario), decide si continúa la sesión activa o abre una nueva, y calcula `error_abs = |value - realValue|`.
5. **Almacenamiento:** INSERT en `samples` y actualización de los contadores de `runs`; COMMIT.
6. **Visualización:** después del COMMIT se emiten eventos `sample`, `run_start`, `run_end` por WebSocket filtrados por rol; el dashboard los aplica en lotes cada 100 ms.

Identificación del usuario de cada cliente

La identidad se establece en tres niveles: (1) el **topic** contiene el `userId` numérico del dueño; (2) el **broker** autentica con `DEVICE_ID/DEVICE_TOKEN` y la ACL impide publicar bajo otro `userId`; (3) el **servidor** comprueba de nuevo que el `deviceId` del payload pertenece a ese usuario (defensa en profundidad). El token se genera al vincular el ESP32 desde la web (`nanoid(32)`), se muestra una sola vez y solo se guarda su hash (bcrypt en BD, PBKDF2-SHA512 en `passwd`). En la web, cada usuario se identifica con JWT (12 h), que se revalida contra la BD en cada petición.

Número de registro (n)

n es el campo `iteration` del payload: entero positivo que el cliente incrementa en cada término (1, 2, 3, ...). Se almacena en `samples.iteration` y cumple dos funciones: indica cuántos términos lleva la aproximación (eje X de los gráficos) y permite **inferir sesiones**: si llega una muestra de la misma serie con n menor o igual al último registrado, el servidor entiende que el cliente reinició y abre una sesión nueva. `runs.last_iteration` guarda el último n de cada sesión.

4. Diseño de la base de datos

Nombre y tecnología

Base de datos **iot** en **PostgreSQL 16** (imagen `postgres:16-alpine` en Docker Compose). El esquema está en `apps/api/db/schema.sql` y se aplica con `pnpm db:reset`, que además carga datos de demostración generados con las mismas funciones de las series.

Tablas, campos y tipos

Tabla / campo	Tipo	Restricciones y propósito
users — usuarios de la plataforma (identificador del cliente)		

id	SERIAL	PRIMARY KEY; es el userId del topic MQTT
email	TEXT	UNIQUE, NOT NULL
password	TEXT	NOT NULL; hash bcrypt
name	TEXT	NOT NULL
role	TEXT	NOT NULL, CHECK ('admin', 'client')
is_active	BOOLEAN	NOT NULL DEFAULT true
created_at	TIMESTAMPZ	NOT NULL DEFAULT now()
devices — ESP32 vinculado a un cliente		
id	TEXT	PRIMARY KEY (ej. esp32-alvaro01); usuario MQTT
user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE
name	TEXT	NOT NULL; alias legible
token_hash	TEXT	NOT NULL; hash bcrypt del token
is_revoked	BOOLEAN	NOT NULL DEFAULT false
created_at, last_seen_at	TIMESTAMPZ	alta y última muestra recibida
runs — sesión: tramo continuo de envío de UNA serie		
id	BIGSERIAL	PRIMARY KEY
user_id	INTEGER	NOT NULL, FK → users(id) ON DELETE CASCADE
device_id	TEXT	FK → devices(id) ON DELETE SET NULL; NULL si viene del simulador
series_key	TEXT	NOT NULL; tipo de serie (pi_leibniz, wallis_pi, basel, ...)
status	TEXT	NOT NULL DEFAULT 'active', CHECK ('active', 'finished')
started_at, ended_at	TIMESTAMPZ	inicio (NOT NULL) y fin de la sesión
sample_count, last_iteration	INTEGER	NOT NULL DEFAULT 0; resumen
last_value, last_error_abs	DOUBLE PRECISION	último valor y error
source	TEXT	NOT NULL, CHECK ('web', 'device')
samples — cada valor generado		
id	BIGSERIAL	PRIMARY KEY
run_id	BIGINT	NOT NULL, FK → runs(id) ON DELETE CASCADE
iteration	INTEGER	NOT NULL; número de registro n
value	DOUBLE PRECISION	NOT NULL; valor generado

error_abs	DOUBLE PRECISION	NOT NULL; error asociado $ x_n - x_{\text{real}} $
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now(); hora del servidor

Con esto cada muestra queda asociada a: **usuario** (samples → runs.user_id), **número de registro** (iteration), **tipo de serie** (runs.series_key), **valor** (value) y **error** (error_abs).

Restricciones y reglas aplicadas

- **PRIMARY KEY** en todas las tablas; SERIAL/BIGSERIAL como autoincremento (BIGSERIAL en runs y samples por el alto volumen de muestras).
- **FOREIGN KEY** con ON DELETE CASCADE (al borrar un usuario se borran sus dispositivos, sesiones y muestras) y ON DELETE SET NULL en runs.device_id (se conserva el historial aunque se elimine la placa).
- **NOT NULL, UNIQUE** (email) y **CHECK** para dominios cerrados (role, status, source).
- **Índice único parcial** uniq_active_run_per_user ON runs(user_id) WHERE status = 'active': la base de datos garantiza que un cliente tenga **como máximo una sesión activa**, incluso con muestras concurrentes.
- **Índices** idx_samples_run (run_id, iteration), idx_runs_user (user_id, started_at DESC) e idx_devices_user para las consultas del dashboard.

Justificación de campos y tipos

- DOUBLE PRECISION coincide con el double de 64 bits del ESP32 y de JavaScript: se almacena el valor sin pérdida de precisión (se muestran 10 decimales).
- El error se guarda ya calculado (error_abs) para que las consultas y gráficos no dependan de recalcularlo y para registrar el valor real vigente al momento de la muestra.
- series_key como TEXT validado en la aplicación (zod) contra la lista compartida permite agregar series sin migraciones.
- TIMESTAMPTZ guarda instantes en UTC y el dashboard los presenta en la hora local.
- Los resúmenes en runs (sample_count, last_value, ...) evitan agregaciones costosas sobre miles de muestras al listar sesiones.
- Separar runs de samples normaliza los datos: los atributos de la sesión (usuario, serie, origen) no se repiten en cada muestra.

Diagrama entidad-relación

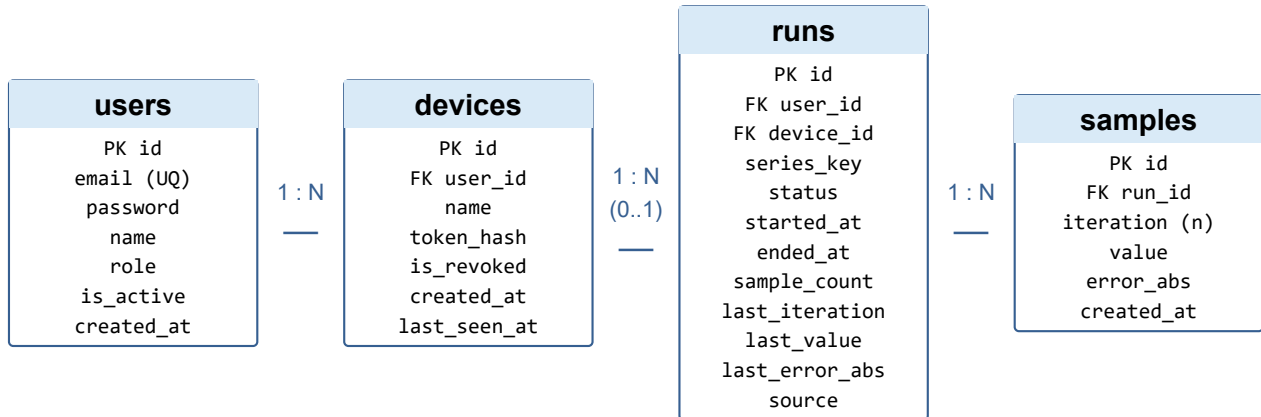


Figura 2: Diagrama entidad-relación. Además, *users* 1 : N *runs* (un usuario tiene muchas sesiones).

5. Desarrollo de la aplicación

Tecnologías, librerías y herramientas

Capa	Tecnologías
Cliente IoT	ESP32-S3, Arduino (WiFi.h, PubSubClient), double y %.10f
Broker	Eclipse Mosquitto 2 (Docker), passwd + acl gestionados por la API, recarga con SIGHUP
Servidor	Node.js 22, TypeScript, Express 5, mqtt.js 5, pg 8, ws 8, zod, jsonwebtoken, bcrypt, nanoid, tsx
Base de datos	PostgreSQL 16 (Docker), extensión pgcrypto para el seed
Dashboard	React 18, Vite 7, TypeScript, Tailwind CSS 4, Recharts 2, React Router 6, date-fns
Herramientas	pnpm workspaces, Docker Compose, GitHub Actions (typecheck + build + deploy por rsync/SSH a un VPS)

Estructura del monorepo: packages/shared (series, precisión, topics: única fuente de verdad), apps/api (servidor), apps/web (dashboard) y mosquitto/ (configuración del broker).

Implementación del cliente

Firmware ESP32. Se conecta a WiFi, se autentica en el broker con `DEVICE_ID` como usuario y `DEVICE_TOKEN` como contraseña (obtenidos en la página **Mi dispositivo**), y en cada ciclo publica la iteración siguiente. La identificación del usuario va en el topic (`telemetry/{userId}/...`) y la del dispositivo en `deviceId`:

```
#define DEVICE_ID    "esp32-alvaro01"
#define DEVICE_TOKEN "....." // se muestra una sola vez al vincular
#define MQTT_TOPIC  "telemetry/2/pi_leibniz"

void connectAll() {
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) delay(300);
  mqtt.setServer(MQTT_HOST, MQTT_PORT);
```

```

while (!mqtt.connected()) {
  if (mqtt.connect(DEVICE_ID, DEVICE_ID, DEVICE_TOKEN)) break; // usuario = id, clave = token
  delay(1000);
}

void loop() {
  if (!mqtt.connected()) connectAll();
  mqtt.loop();
  static int k = 0; static double acc = 0;
  k++;
  acc += 4.0 * ((k % 2) ? 1 : -1) / (2 * k - 1); // Leibniz
  char buf[128];
  snprintf(buf, sizeof buf, "{\"iteration\":%d,\"value\":%.10f,\"deviceId\":\"%s\"}", k, acc,
  DEVICE_ID);
  mqtt.publish(MQTT_TOPIC, buf);
  delay(1000);
}

```

Simulador web (apps/api/src/simulator.ts). Para clientes sin placa: un temporizador por usuario calcula la serie con la función compartida y **publica por MQTT** como un dispositivo (sin deviceId), de modo que sus datos pasan por el mismo broker, validación, sesiones y base de datos:

```

const timer = setInterval(() => {
  k++;
  const r = computeTerm(seriesKey, k, acc);
  acc = r.acc;
  const payload = { iteration: k, value: Number(r.value.toFixed(10)), ts: Math.floor(Date.now() /
  1000) };
  getMqtt()?.publish(telemetryTopic(userId, seriesKey), JSON.stringify(payload));
}, intervalMs); // 50 - 10 000 ms, validado con zod

```

Vinculación del dispositivo (routes/devices.ts): genera id = esp32-xxxxxxx y token = nanoid(32), guarda el hash bcrypt, escribe credenciales y ACL en Mosquitto y lo recarga; si el broker falla, deshace el INSERT.

Implementación del servidor

El puente MQTT (apps/api/src/mqtt/bridge.ts) recibe, valida y encola cada mensaje. Nada que llegue por MQTT puede tumbar el proceso: los errores se registran y la muestra se descarta. La reconexión usa **backoff** exponencial de 1 s a 30 s.

```

const payloadSchema = z.object({
  iteration: z.number().int().positive().max(2_147_483_647),
  value: z.number().finite(),
  deviceId: z.string().min(1).max(64).optional(),
  ts: z.number().optional(),
});

c.on('message', (topic, buf) => {
  const parsedTopic = parseTelemetryTopic(topic); // /^telemetry\/(\d+)\/([a-z0-9_]+)$/
  if (!parsedTopic || !isSeriesKey(parsedTopic.seriesKey)) return;
  let json; try { json = JSON.parse(buf.toString('utf8')); } catch { return; }
  const p = payloadSchema.safeParse(json);
  if (!p.success) return;
  const { userId, seriesKey } = parsedTopic;

```

```
enqueue(userId, () => ingestSample({ userId, seriesKey, iteration: p.data.iteration,
  value: p.data.value, deviceId: p.data.deviceId ?? null })); // una cola por usuario
});
```

La cola por usuario evita que la iteración 6 se confirme antes que la 5 (lo que parecería un reinicio y partiría la sesión). Además de MQTT, el servidor expone una API REST (/api/auth, /api/admin/*, /api/client/*, /api/devices, /api/simulate/*) protegida con JWT y roles, y un **barredor** que cada 30 s cierra las sesiones sin muestras durante 2 minutos.

Implementación de las tres series matemáticas

Las series se definen una sola vez en packages/shared/src/series.ts y las usan el simulador, el seed y el dashboard. computeTerm es una función pura: recibe el término k y el acumulador, y devuelve el nuevo acumulador y el valor a reportar.

```
export function initialAcc(key: SeriesKey): number {
  return key === 'wallis_pi' ? 1 : 0; // producto inicia en 1, sumas en 0
}

export function computeTerm(key: SeriesKey, k: number, acc: number) {
  const sign = k % 2 === 1 ? 1 : -1; // (-1)^(k+1)
  const odd = 2 * k - 1;
  switch (key) {
    case 'pi_leibniz': //  $\pi = \sum 4(-1)^{(k+1)}/(2k-1)$ 
      acc += (4 * sign) / odd;
      return { acc, value: acc };
    case 'wallis_pi': //  $\pi = 2 \prod (2k/(2k-1)) \cdot (2k/(2k+1))$ 
      acc *= ((2 * k) / odd) * ((2 * k) / (2 * k + 1));
      return { acc, value: 2 * acc };
    case 'basel': //  $\pi^2/6 = \sum 1/k^2$ 
      acc += 1 / (k * k);
      return { acc, value: acc };
    // ... ln2_alternating, euler_gamma, pi2_over_8, catalan
  }
}
```

El error se calcula en el servidor con el valor real de cada serie (realValue, con 15 decimales): $e_n = |x_n - x_{\text{real}}|$; el dashboard muestra también el error relativo $\frac{e_n}{x_{\text{real}}} \cdot 100\%$. El paquete incluye autocorrobaciones (series.check.ts) que se ejecutan en CI con pnpm lint.

Inserción de datos en la base de datos

La conexión se hace con un pg.Pool configurado por DATABASE_URL. El helper tx ejecuta una función dentro de BEGIN ... COMMIT y hace ROLLBACK ante cualquier error:

```
export async function tx<T>(fn: (c: pg.PoolClient) => Promise<T>): Promise<T> {
  const c = await pool.connect();
  try { await c.query('BEGIN'); const r = await fn(c); await c.query('COMMIT'); return r; }
  catch (e) { await c.query('ROLLBACK').catch(() => {}); throw e; }
  finally { c.release(); }
}
```

La ingesta (apps/api/src/runs.ts) bloquea la fila del usuario, valida el dispositivo, decide la sesión e inserta la muestra con consultas parametrizadas (\$1, \$2..., sin concatenar entradas):

```

const user = await c.query('SELECT is_active FROM users WHERE id = $1 FOR UPDATE', [userId]);
if (!user.rows[0]?.is_active) throw new DiscardSample(`usuario ${userId} no válido`);
// ... dispositivo existente, no revocado y del mismo usuario
const continues = active && active.series_key === seriesKey
  && active.device_id === deviceId && iteration > active.last_iteration;
if (!continues) {
  if (active) events.push(...(await closeRuns(c, 'r.id = $1', [active.id]]));
  const created = await c.query(
    `INSERT INTO runs (user_id, device_id, series_key, source) VALUES ($1,$2,$3,$4) RETURNING id`,
    [userId, deviceId, seriesKey, deviceId ? 'device' : 'web']);
  runId = created.rows[0].id;
}
const errorAbs = Math.abs(value - seriesByKey[seriesKey].realValue);
await c.query(`INSERT INTO samples (run_id, iteration, value, error_abs) VALUES ($1,$2,$3,$4)`,
  [runId, iteration, value, errorAbs]);
await c.query(`UPDATE runs SET sample_count = sample_count + 1, last_iteration = $2,
  last_value = $3, last_error_abs = $4 WHERE id = $1`, [runId, iteration, value, errorAbs]);

```

Manejo de errores: las reglas de negocio lanzan DiscardSample (se registra y se descarta); si el índice único parcial detecta dos sesiones activas (p. ej. dos instancias de la API), se captura la violación de unicidad y la transacción se reintenta una vez; los eventos WebSocket se emiten **solo después del COMMIT**, para no anunciar datos que luego se deshagan.

Lectura o recuperación de datos

Las sesiones se listan con filtros opcionales (usuario, serie, estado, rango de fechas) construidos de forma parametrizada. Para clientes, las columnas numéricas ni siquiera se seleccionan (el cliente ve su historial pero no los valores):

```

export async function listRuns(f: RunFilters, numeric: boolean) {
  const where: string[] = [], params: unknown[] = [];
  const add = (sql: string, v: unknown) => { params.push(v); where.push(sql.replace('?', ` ${params.length}`)); };
  if (f.userId) add('r.user_id = ?', f.userId);
  if (f.seriesKey) add('r.series_key = ?', f.seriesKey);
  if (f.from) add('coalesce(r.ended_at, now()) >= ?', f.from);
  if (f.to) add('r.started_at < ?', f.to);
  const { rows } = await pool.query(
    `SELECT ${BASE_COLUMNS}${numeric ? NUMERIC_COLUMNS : ''}
    FROM runs r JOIN users u ON u.id = r.user_id LEFT JOIN devices d ON d.id = r.device_id
    ${where.length ? 'WHERE ' + where.join(' AND ') : ''}
    ORDER BY (r.status = 'active') DESC, r.started_at DESC LIMIT 2000`, params);
  return rows;
}

```

Las muestras de un usuario y serie se obtienen con GET /api/admin/users/:id/series/:key (ordenadas por sesión e iteración) y las de una sesión, paginadas, con GET /api/admin/runs/:runId/samples?limit&offset.

Otras funciones importantes

- **Autenticación y roles:** registro e inicio de sesión (bcrypt + JWT); el administrador gestiona usuarios (crear, editar rol, desactivar, restablecer contraseña, borrar datos). Al desactivar un usuario se detiene su simulador, se cierran sus sesiones y se cortan sus WebSockets.

- **Gestión de credenciales del broker** (`broker-credentials.ts`): genera el `passwd` con el mismo formato que `mosquitto_passwd` (`$7$, PBKDF2-SHA512`) y regenera la ACL desde la BD; recarga Mosquitto sin cortar conexiones.
- **Revocación de dispositivos**: marca `is_revoked`, cierra su sesión y quita sus credenciales del broker.
- **WebSocket filtrado por rol** (`ws.ts`): el administrador recibe todos los eventos; el cliente solo los de sus sesiones y sin campos numéricos (lista blanca). Keepalive ping/pong cada 30 s.
- **Despliegue continuo**: GitHub Actions ejecuta typecheck y autocomprobaciones, compila la web y despliega en un VPS (`iot-dashboard.tumype.com`).

6. Desarrollo del dashboard

Diseño y herramientas

El dashboard (`apps/web`) es una SPA en **React 18 + Vite + TypeScript** con **Tailwind CSS** y gráficos **Recharts**, sin librerías de componentes prefabricados. Tiene dos perfiles:

- **Administrador**: *Panel general* (indicadores y quién envía ahora), *Usuarios*, *Detalle de usuario* (historial en tabla o línea de tiempo, gráficos y muestras por sesión) y *Sesiones* (todas, con filtros).
- **Cliente**: *Mi dispositivo* (vincular/revocar ESP32), *Mis envíos* (línea de tiempo e historial) y *Simulador*.

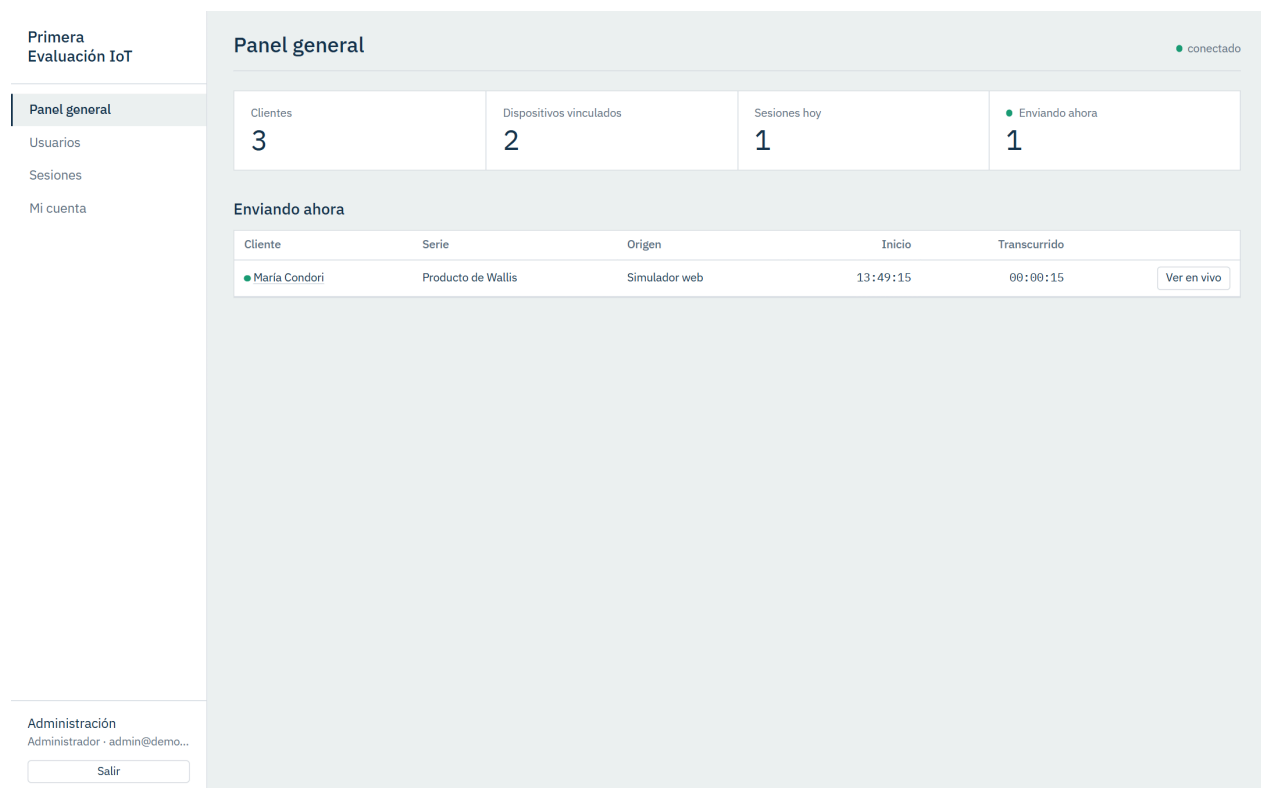


Figura 3: Panel general del administrador: clientes, dispositivos, sesiones del día y envíos en curso.

Visualización de datos por usuario y su error

Al abrir una sesión desde el detalle de un usuario se muestra un diálogo con los datos de la sesión (cliente, serie, origen, inicio, fin, duración, iteraciones, estado), el gráfico de la serie y la tabla de muestras con **iteración n**, **valor aproximado**, **valor real**, **error absoluto**, **error relativo** y **hora**. Los dígitos que ya coinciden con el valor real se resaltan en oscuro y el resto en gris, lo que permite leer de un vistazo cuántos decimales son correctos.

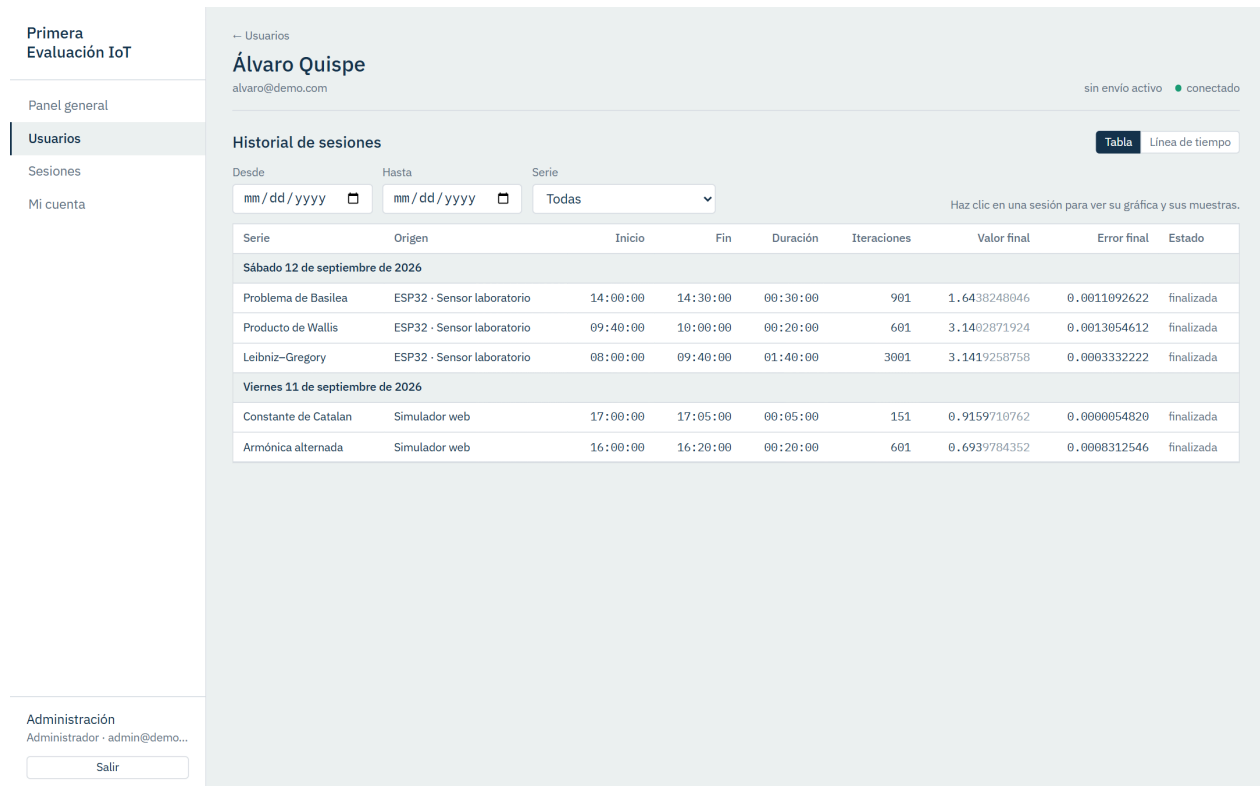


Figura 4: Detalle del usuario Álvaro Quispe: historial de sesiones con valor y error final, agrupado por día.

Gráficos

Cada gráfico tiene dos modos: **valor** (aproximación frente a iteración, con una línea discontinua en el valor real y una **franja roja de error** que rellena la distancia entre la aproximación y el valor real) y **error log** (error absoluto en escala logarítmica, donde la tasa de convergencia se ve como pendiente). Sin sesión seleccionada se dibuja una línea por sesión, más clara cuanto más antigua. Para mantener fluidez, cada gráfico se limita a 2000 puntos con un submuestreo que conserva los puntos recientes intactos y resume los antiguos en cubos **mínimo/máximo** (un muestreo «uno de cada N» aplanaría la oscilación de Leibniz).

Visualización del error en los gráficos. En la primera versión el error solo se representaba con la franja roja del modo valor. Al revisar las capturas se comprobó que **no se veía**: el eje de valores abarca todo el recorrido de la serie (p. ej. de 2.4 a 4.0 en Leibniz), mientras que después de unas pocas iteraciones el error es del orden de 10^{-3} – 10^{-4} , es decir, menos de un píxel de alto. La franja solo aparecía en las primeras iteraciones y el resto del gráfico parecía no tener error. Por eso se modificó `SeriesChart.tsx`:

- En modo **valor** se añadió un **segundo eje Y a la derecha, en rojo y en escala logarítmica**, y sobre él una **curva roja con el error absoluto** $e_n = |x_n - x_{\text{real}}|$ de cada iteración. Así, en

el mismo gráfico se ve la aproximación (azul, eje izquierdo) acercándose al valor real (línea discontinua) y el error (rojo, eje derecho) bajando durante **todas** las iteraciones, no solo al inicio. Se conserva la franja roja entre la curva y el valor real.

- En modo **error log** la curva del error se dibuja también en rojo, para que el color identifique siempre al error.
- El **tooltip** muestra en filas separadas el valor de la sesión y su error en la iteración señalada (por ejemplo, sesión #4 : 3.1354387190 y error #4 : 0.0061539346).
- Ambos ejes logarítmicos usan el mismo dominio (potencias de 10 que cubren el error mínimo y máximo) y el error 0 exacto se limita a 10^{-16} , porque la escala log no admite 0.



Figura 5: Sesión 4 (Producto de Wallis), modo valor: curva azul de la aproximación hacia π y curva roja del error en el eje logarítmico derecho, que baja durante las 601 iteraciones hasta 1.31×10^{-3} .



Figura 6: Sesión 5 (Leibniz-Gregory, ESP32), modo valor: la aproximación oscila alrededor de π y la curva roja muestra el error bajando de forma continua hasta 3.33×10^{-4} en $n = 3001$, algo que con solo la franja no se apreciaba.

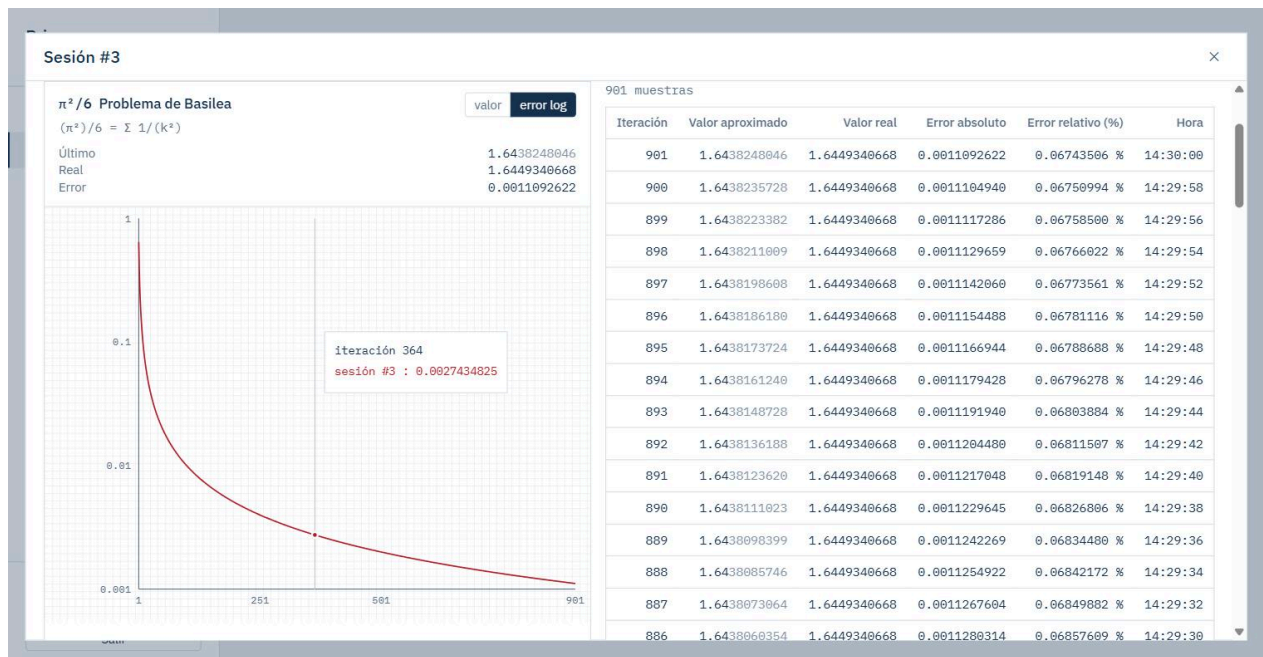


Figura 7: Sesión 3 (Problema de Basilea), modo error log: error absoluto en rojo en escala logarítmica, decreciente de 0.64 a 0.0011 en 901 iteraciones.

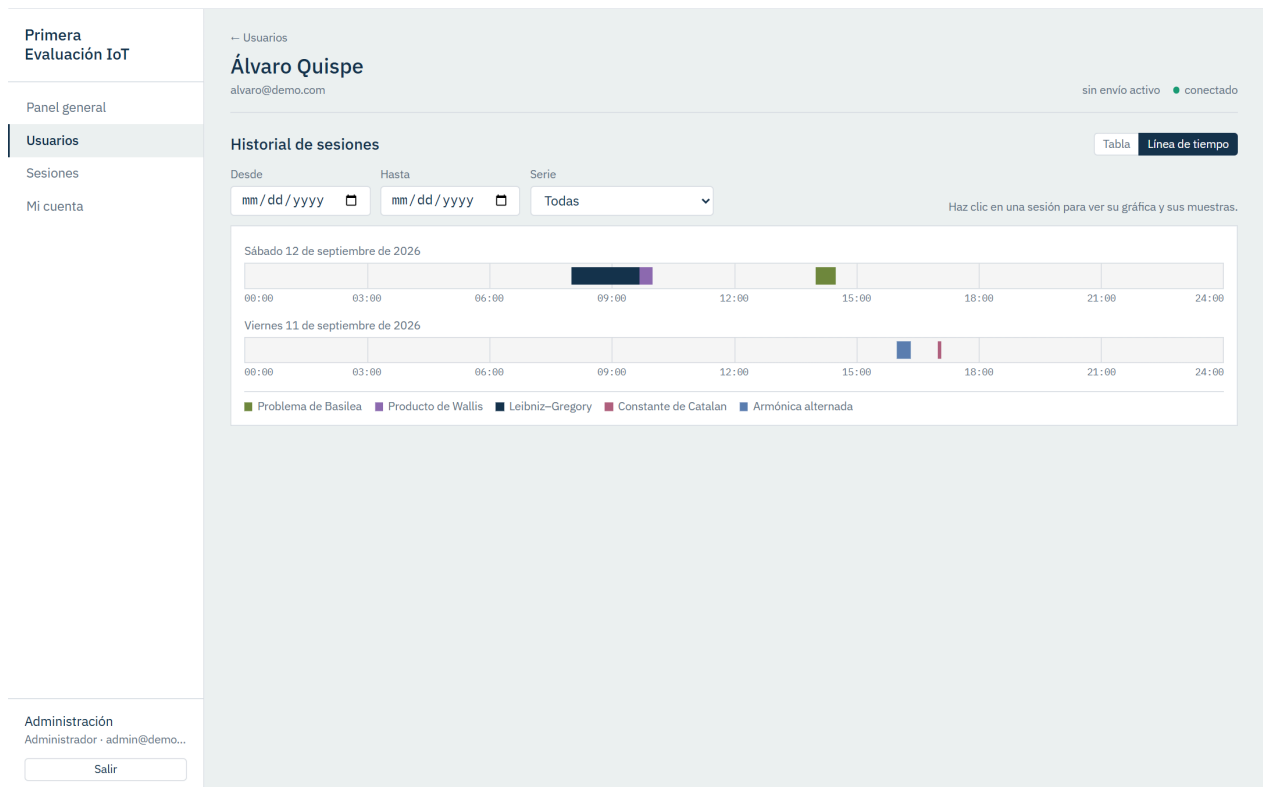


Figura 8: Vista de línea de tiempo: sesiones de cada día coloreadas por serie; permite ver el cambio de Leibniz a Wallis a las 09:40.

Código relevante del dashboard

Gráfico con escala conmutable (components/SeriesChart.tsx):

```
<ComposedChart margin={{ top: 12, right: 16, bottom: 4, left: 4 }}>
  <XAxis type="number" dataKey="iteration" domain={['dataMin', 'dataMax']} />
  {mode === 'value'
    ? <YAxis type="number" domain={['auto', 'auto']} tickFormatter={(v) => v.toFixed(6)} />
    : <YAxis type="number" scale="log" domain={logDomain} ticks={logTicks} tickFormatter={fixedTick} />
  }
  {/* eje derecho: error en escala log, visible aunque en el eje de valores mida menos de 1 px */}
  {mode === 'value' && <YAxis yAxisId="err" orientation="right" scale="log" domain={logDomain}
    stroke="#B3202C" />}
  {mode === 'value' && <ReferenceLine y={series.realValue} strokeDasharray="4 3" />}
  <Tooltip formatter={(v, _n, item) => (mode === 'value' && item.dataKey === 'y' ? fmt.value(v) :
    fmt.errorAbs(v))}
    labelFormatter={(l) => `iteración ${l}`} />
  {/* franja de error: cada punto lleva band = [valor real, valor aproximado] */}
  {mode === 'value' && lines.map((l) => (
    <Area key={`err-${l.runId}`} data={l.data} dataKey="band" stroke="none" fill="#B3202C"
    fillOpacity={0.15} />
  ))}
  {/* curva del error: cada punto lleva e = max(|x_n - x_real|, 1e-16) */}
  {mode === 'value' && lines.map((l) => (
    <Line key={`e-${l.runId}`} yAxisId="err" data={l.data} dataKey="e" stroke="#B3202C" dot={false} />
  ))}
  {lines.map((l) => (
    <Line key={l.runId} data={l.data} dataKey="y" stroke={mode === 'value' ? '#16324F' : '#B3202C'}
    strokeOpacity={l.opacity} dot={l.data.length <= 300} />
  ))}
</ComposedChart>
```

```
    )}}  
</ComposedChart>
```

Submuestreo que preserva la forma de la curva (lib/downsample.ts):

```
export function downsample<T>(points: T[], max: number, y: (p: T) => number): T[] {  
  if (points.length <= max) return points;  
  const keepRecent = Math.floor(max / 2);  
  const old = points.slice(0, points.length - keepRecent), recent = points.slice(-keepRecent);  
  const buckets = Math.max(1, Math.floor((max - keepRecent) / 2)), size = old.length / buckets;  
  const out: T[] = [];  
  for (let b = 0; b < buckets; b++) {  
    const start = Math.floor(b * size), end = Math.min(old.length, Math.floor((b + 1) * size));  
    let lo = start, hi = start;  
    for (let i = start + 1; i < end; i++) { if (y(old[i]) < y(old[lo])) lo = i; if (y(old[i]) >  
y(old[hi])) hi = i; }  
    if (lo === hi) out.push(old[lo]); else out.push(old[Math.min(lo, hi)], old[Math.max(lo, hi)]);  
  }  
  return out.concat(recent);  
}
```

Actualización en tiempo real

No se usa **polling**. La web abre un único WebSocket por pestaña (/ws?token=JWT) con reconexión automática. El hook useLiveSamples evita huecos y duplicados entre histórico y tiempo real:

1. Se suscribe al WebSocket y **encola** todo lo que llega.
2. Carga el histórico por REST (sesiones y muestras de las series del usuario).
3. Sustituye el estado por el histórico y aplica la cola, **deduplicando por sample.id**.
4. Aplica nuevos eventos al estado de React **en lotes cada 100 ms** (actualizar por mensaje congelaría la interfaz con intervalos de 50 ms).
5. En cada reconexión repite la carga, porque durante el corte pudieron perderse eventos.

La frecuencia efectiva la define el cliente (1 s en el firmware de ejemplo; 50 ms a 10 s en el simulador) y el dashboard la refleja con una latencia máxima de 100 ms.

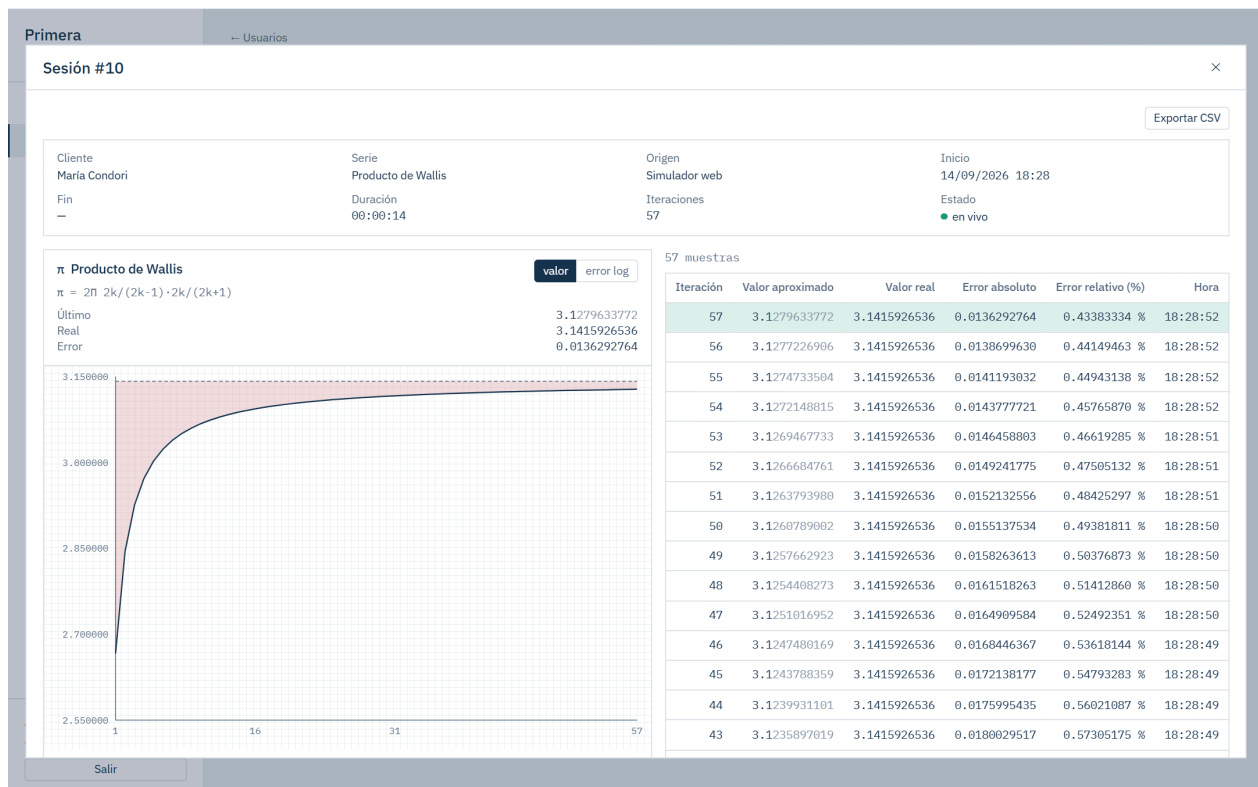


Figura 9: Sesión en vivo de María Condori (simulador, Producto de Wallis cada 250 ms): la tabla, el gráfico y su franja de error crecen en tiempo real.

Interactividad

- **Selección de usuario** (lista de usuarios → detalle) y **selección de sesión** (clic en la tabla o en la línea de tiempo).
- **Filtros** por rango de fechas (desde/hasta), serie, cliente y estado (activa/finalizada).
- Conmutadores **Tabla / Línea de tiempo** y **valor / error log**; **tooltip** con valor exacto y error absoluto por iteración.
- **Exportar CSV** de las muestras de una sesión (iteración, valor, error absoluto, fecha), compatible con Excel (UTF-8 con BOM).
- Botón **Ver en vivo** en el panel general para abrir la sesión en curso.

Primera Evaluación IoT

Panel general

Usuarios

Sesiones

Mi cuenta

Sesiones

Todas las sesiones de todos los clientes.

conectado

Cliente

Serie

Estado

Desde

Hasta

Todos

Todos

Todos

mm/dd/yyyy

mm/dd/yyyy

Cliente	Serie	Origen	Inicio	Fin	Duración	Iteraciones	Valor final	Error final	Estado
En curso									
María Condori	Producto de Wallis	Simulador web	13:49:15	—	00:00:21	75	3.1312073086	0.0103853450	en curso
Sábado 12 de septiembre de 2026									
Álvaro Quispe	Problema de Basilea	ESP32 - Sensor laboratorio	14:00:00	14:30:00	00:30:00	901	1.6430240046	0.0011992622	finalizada
Álvaro Quispe	Producto de Wallis	ESP32 - Sensor laboratorio	09:40:00	10:00:00	00:20:00	601	3.1402071924	0.0013054612	finalizada
Álvaro Quispe	Leibniz–Gregory	ESP32 - Sensor laboratorio	08:00:00	09:40:00	01:40:00	3001	3.1419250758	0.0003332222	finalizada
Viernes 11 de septiembre de 2026									
Álvaro Quispe	Constante de Catalan	Simulador web	17:00:00	17:05:00	00:05:00	151	0.9159710762	0.0000054820	finalizada
Álvaro Quispe	Armónica alternada	Simulador web	16:00:00	16:20:00	00:20:00	601	0.6939704352	0.0008312546	finalizada
María Condori	Leibniz–Gregory	Simulador web	11:00:00	11:15:00	00:15:00	451	3.1430099450	0.0022172922	finalizada

Administración

Administrador - admin@demo...

Salir

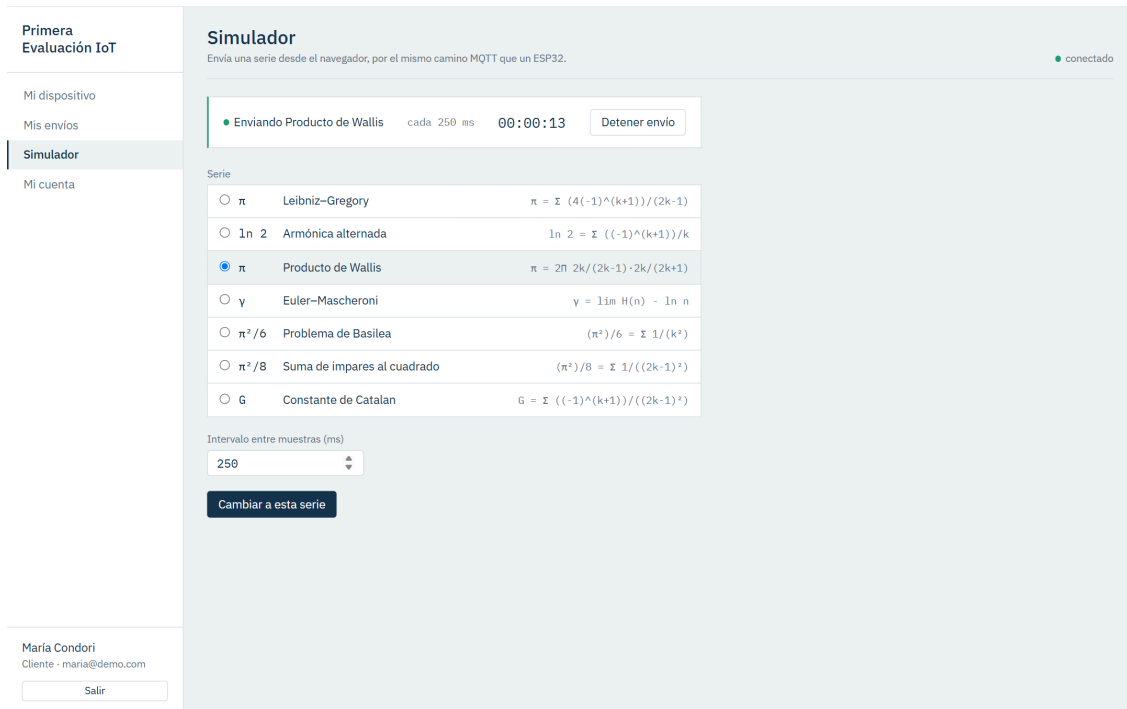


Figura 11: Simulador web del cliente enviando el Producto de Wallis cada 250 ms.

Generación y almacenamiento de las tres series

Sesiones registradas en la tabla runs (consulta directa con psql; horas en UTC):

name	id	series_key	source	status	sample_count	last_value	last_error_abs
Álvaro Quispe	1	catalan	web	finished	151	0.9159710762	5.482022780e-06
Álvaro Quispe	2	ln2_alternating	web	finished	601	0.6939784352	0.000831254640
Álvaro Quispe	3	basel	device	finished	901	1.6438248046	0.001109262248
Álvaro Quispe	4	wallis_pi	device	finished	601	3.1402871924	0.001305461190
Álvaro Quispe	5	pi_leibniz	device	finished	3001	3.1419258758	0.000333222210
María Condori	6	pi_leibniz	web	finished	451	3.1438099458	0.002217292210

Registros asociados a cada usuario y al valor n

Muestras de la tabla samples unidas con runs (usuario 2, serie de Basilea), donde n es la iteración:

run_id	user_id	series_key	n	value	error_abs
3	2	basel	1	1	0.644934066848226
3	2	basel	2	1.25	0.394934066848226
3	2	basel	3	1.3611111111	0.283822955748226
3	2	basel	10	1.5497677312	0.095166335648226
3	2	basel	100	1.6349839002	0.009950166648226

Desde la vista del cliente, cada usuario ve solo su propio historial (sin valores numéricos), y el administrador ve el de todos:

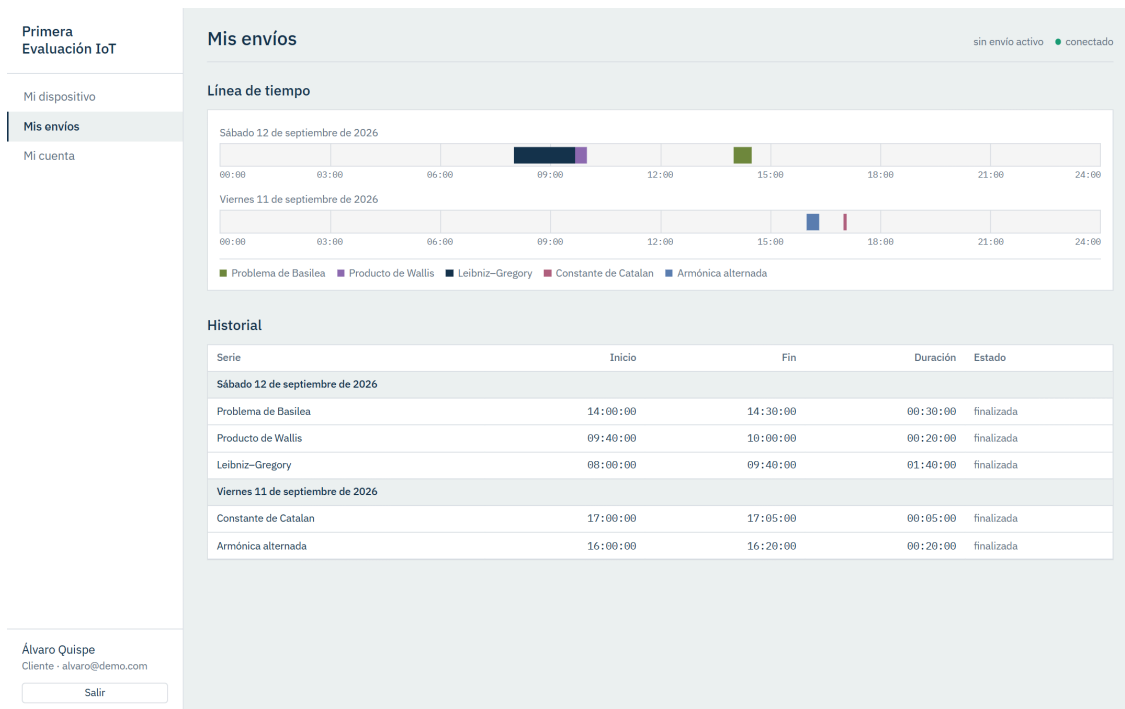


Figura 12: Vista «Mis envíos» del cliente Álvaro: línea de tiempo e historial propios.

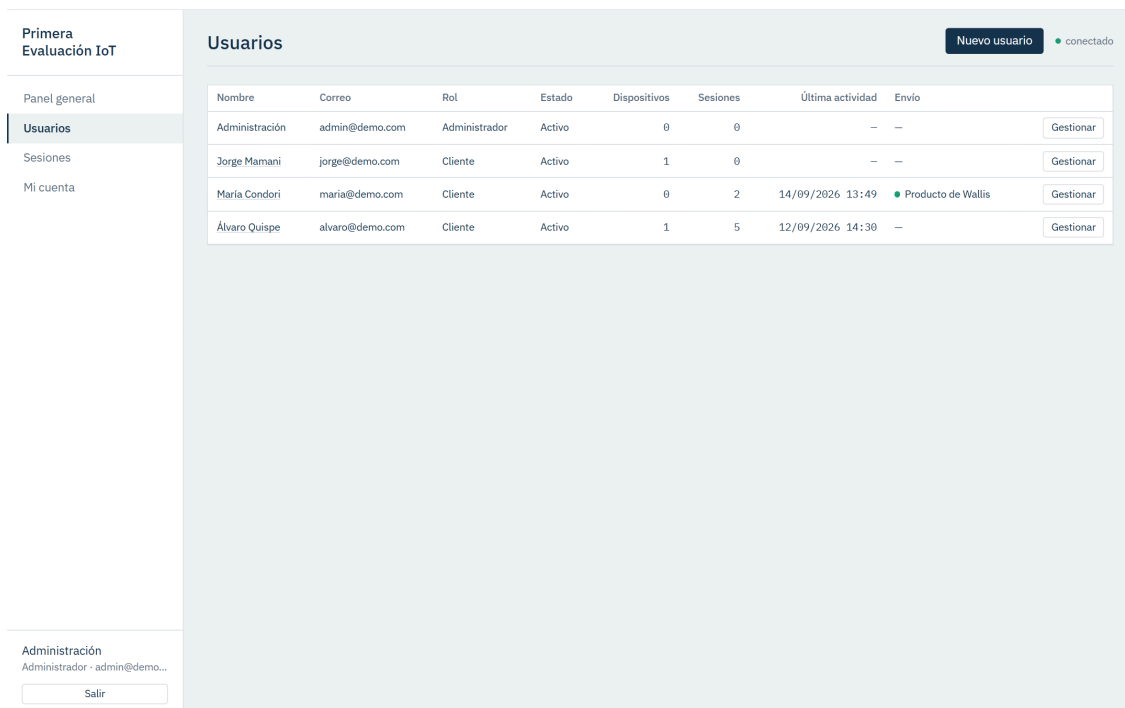


Figura 13: Listado de usuarios del administrador con dispositivos, sesiones y muestras por usuario.

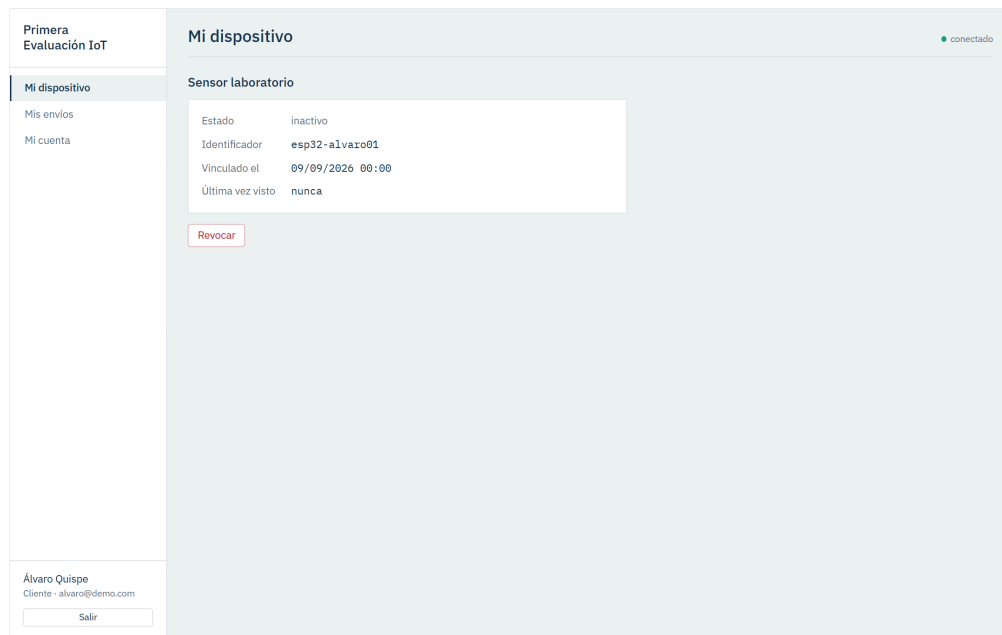


Figura 14: Página «Mi dispositivo»: ESP32 vinculado esp32-alvaro01, con opción de revocar.

Capturas del dashboard con datos y error

Las capturas se tomaron de nuevo tras añadir la curva roja del error con su eje logarítmico. Ahora el error se ve claramente en toda la sesión: en modo valor, la curva roja (eje derecho) baja mientras la aproximación se acerca al valor real (figuras de Wallis y Leibniz); en modo error log se ve el mismo decrecimiento a pantalla completa. La lectura de la curva coincide con la columna **Error absoluto** de la tabla contigua.

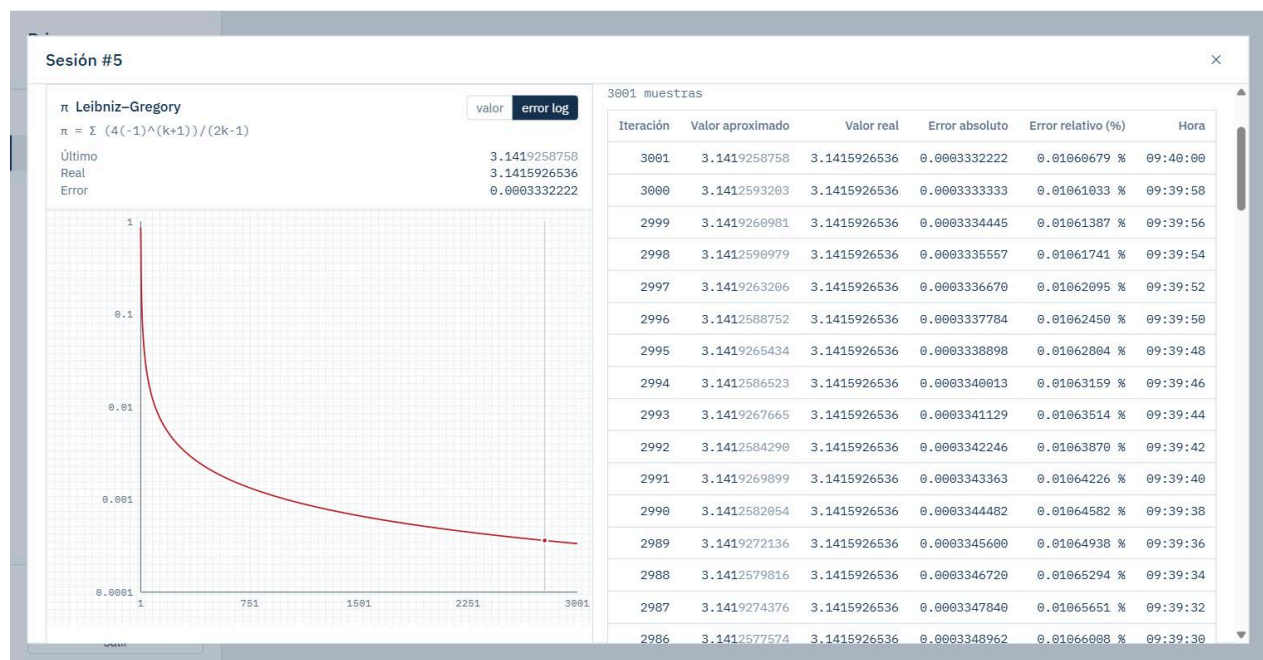


Figura 15: Sesión de Leibniz en modo error logarítmico: la curva roja baja de 0.86 a 3.33×10^{-4} en 3001 iteraciones, casi una recta de pendiente -1 (error $\approx 1/n$).

Análisis de resultados

Serie	n	Valor final	Error absoluto	Observación
Leibniz–Gregory	3001	3.1419258758	3.33×10^{-4}	Error $\approx 1/n$; oscila alrededor de π (3 decimales correctos)
Producto de Wallis	601	3.1402871924	1.31×10^{-3}	Error $\approx \pi/(4n)$; se acerca por debajo
Problema de Basilea	901	1.6438248046	1.11×10^{-3}	Error $\approx 1/n$; crece monótonamente hacia $\pi^2/6$

- Las tres series convergen con orden $O(\frac{1}{n})$: para ganar un decimal se necesitan unas 10 veces más iteraciones. En la escala logarítmica esto se ve como una recta de pendiente constante (Figura de Basilea).
- Los errores medidos concuerdan con la teoría: Leibniz en $n = 3001$ da $\frac{1}{3001} \approx 3.33 \times 10^{-4}$ y Wallis en $n = 601$ da $\frac{\pi}{4 \cdot 601} \approx 1.31 \times 10^{-3}$. Con igual n , Wallis ($\approx \frac{0.785}{n}$) es algo más preciso que Leibniz ($\frac{1}{n}$); además Leibniz oscila, mientras Wallis y Basilea se acercan de forma monótona.
- Como contraste, la constante de Catalan (error $\frac{1}{n^2}$) alcanza 5.5×10^{-6} con solo 151 iteraciones.
- Los valores almacenados coinciden con los calculados teóricamente (por ejemplo, Basilea en $n = 2$: $1 + \frac{1}{4} = 1.25$), lo que valida la cadena de generación, transmisión y almacenamiento sin pérdida de precisión.
- Las sesiones se infirieron correctamente: el cambio de Leibniz a Wallis a las 09:40 cerró una sesión y abrió otra, y el simulador de María generó una sesión propia con origen web.

8. Conclusiones

- Se cumplió el objetivo general: la plataforma genera, transmite, almacena y visualiza las series de Leibniz, Wallis y Basilea (más cuatro adicionales) por usuario, con su número de registro n y su error de aproximación.
- La arquitectura basada en MQTT desacopla los clientes del servidor: el ESP32 y el simulador web usan exactamente el mismo camino, lo que permitió probar el sistema completo sin hardware.
- La identificación del usuario es robusta porque se valida en tres capas (topic, ACL del broker y servidor), y la revocación de un dispositivo no afecta a los demás.
- El diseño relacional (users $\rightarrow$ devices $\rightarrow$ runs $\rightarrow$ samples) con claves foráneas, CHECK e índice único parcial garantiza la integridad aun con muestras concurrentes; el uso de transacciones evita estados inconsistentes.
- El dashboard permite interpretar la convergencia: la curva roja del error sobre el gráfico de valores, la escala logarítmica del error y el resaltado de dígitos correctos muestran de forma inmediata que las tres series convergen como $O(\frac{1}{n})$.
- La actualización en tiempo real por WebSocket, con carga histórica, deduplicación y aplicación por lotes, mantiene la interfaz fluida incluso con intervalos de envío de 250 ms.

9. Anexos y entregables

Repositorio GitHub: https://github.com/alvaro-tc/iot_dashboard

Acceso configurado para la docente: pamela.valenzuela.f@ucb.edu.bo.

- **Código fuente completo:** packages/shared (series), apps/api (servidor, esquema SQL y seed), apps/web (dashboard), mosquitto/ (broker), docker-compose.yml, .github/workflows/deploy.yml; guía de conexión del ESP32 en docs/esp32-mqtt.md.
- **Puesta en marcha:**

```
cp .env.example .env
docker compose up -d      # PostgreSQL (5433) y Mosquitto (1883)
pnpm install
pnpm db:reset              # esquema + seed + credenciales MQTT de demo
pnpm dev                  # API :4000 y web http://localhost:5173
```

- **Credenciales de prueba:** admin@demo.com / admin123 (administrador); alvaro@demo.com, maria@demo.com, jorge@demo.com con contraseña cliente123 (clientes).
- **Capturas de pantalla y diagramas:** incluidos en las secciones 3, 4, 6 y 7; archivos en docs/informe/img/.
- El informe y los archivos generados se comprimen en un archivo **.rar** y se suben al LMS por cada estudiante.
- Exposición y defensa: lunes 07 y viernes 11 de septiembre de 2026, en horario de clases.

Lista de verificación antes de la entrega

Criterio	Verificación
Aplicación cliente-servidor documentada	☒
Tres tipos de series matemáticas implementados y explicados	☒
Identificador de usuario/cliente registrado	☒
Número de registro (n) considerado	☒
Base de datos y diagrama incluidos	☒
Dashboard con datos por usuario y error	☒
Gráficos relevantes incluidos	☒
Repositorio GitHub accesible para la docente	☒
Archivo .rar o .zip preparado para LMS	☒ (.rar)